



JAVA TECHNOLOGIES

SUBJECT CODE : UQ24CA251B

Samyukta D Kumta
Computer Applications

JAVA TECHNOLOGIES



Course Content:

Unit 3: Java Database Connectivity (JDBC)

Annotations: Built-in, JDBC Drivers and Types, Communicating with Database using Java; Database Operations using Java: Understanding CRUD Operations, Classes and Interfaces of JDBC API, Creation and Retrieval of Data using JDBC

Experiential Learning: Developing programs using JDBC and communicate with MySQL Database

Java annotations

Types of Annotations

1. Built-in Annotation
 - Annotations for the compiler
 - Meta-annotations (used to define other annotations)
2. Custom Annotation

JAVA TECHNOLOGIES

Compiler-Level Built-in Annotations:

Annotation	Used On	Description
@Override	Method	Indicates that a method overrides a method from a superclass. Helps catch errors if method signatures don't match.
@Deprecated	Class, Method, Field	Marks a program element as deprecated — means it should not be used anymore. Compiler gives a warning if used.
@SuppressWarnings	Class, Method, Field, Variable	Tells the compiler to suppress specific warnings , like “unchecked” or “deprecation”.
@SafeVarargs	Constructor or Method with varargs	Ensures type safety for variable argument methods that use generics. Prevents unchecked warnings.
@FunctionalInterface	Interface	Ensures the interface has exactly one abstract method , suitable for lambda expressions.

JAVA TECHNOLOGIES



Java Provides five built-in annotations.

1. **@Override** - When we want to override a method of Superclass, we should use this annotation to inform compiler that we are overriding a method. So when superclass method is removed or changed, compiler will show error message. Learn why we should always use [java override annotation](#) while overriding a method.
2. **@Deprecated** - when we want the compiler to know that a method is deprecated, we should use this annotation. Java recommends that in javadoc, we should provide information for why this method is deprecated and what is the alternative to use.

JAVA TECHNOLOGIES



Java Provides five built-in annotations.

3. **@SuppressWarnings** - This is just to tell compiler to ignore specific warnings they produce, for example using raw types in java generics. It's retention policy is SOURCE and it gets discarded by compiler.
4. **@FunctionalInterface** - This annotation was introduced in Java 8 to indicate that the interface is intended to be a functional interface.(SAM: Single Abstract method)
5. **@SafeVarargs** - A programmer assertion that the body of the annotated method or constructor does not perform potentially unsafe operations on its varargs parameter.

JAVA TECHNOLOGIES

Meta-Annotations (used to define your own annotations):

These annotations apply **to other annotations**, controlling how custom annotations behave.

Annotation	Description
@Retention	Specifies how long an annotation is retained (e.g., SOURCE, CLASS, RUNTIME).
@Target	Specifies where an annotation can be used (e.g., METHOD, FIELD, TYPE).
@Documented	Indicates that the annotation should appear in Javadoc .
@Inherited	Allows subclasses to inherit an annotation from a superclass.
@Repeatable	Allows an annotation to be used multiple times on the same element.

There are five types of meta annotations:

1. **@Documented** - indicates that elements using this annotation should be documented by javadoc and similar tools. This type should be used to annotate the declarations of types whose annotations affect the use of annotated elements by their clients. If a type declaration is annotated with Documented, its annotations become part of the public API of the annotated elements.
2. **@Target** - indicates the kinds of program element to which an annotation type is applicable. Some possible values are TYPE, METHOD, CONSTRUCTOR, FIELD etc. If Target meta-annotation is not present, then annotation can be used on any program element.

There are five types of meta annotations:

3. **@Inherited** - indicates that an annotation type is automatically inherited. If user queries the annotation type on a class declaration, and the class declaration has no annotation for this type, then the class's superclass will automatically be queried for the annotation type. This process will be repeated until an annotation for this type is found, or the top of the class hierarchy (Object) is reached.
4. **@Retention** - indicates how long annotations with the annotated type are to be retained. It takes RetentionPolicy argument whose Possible values are SOURCE, CLASS and RUNTIME
5. **@Repeatable** - used to indicate that the annotation type whose declaration it annotates is repeatable.

JAVA TECHNOLOGIES



JAVA Drivers

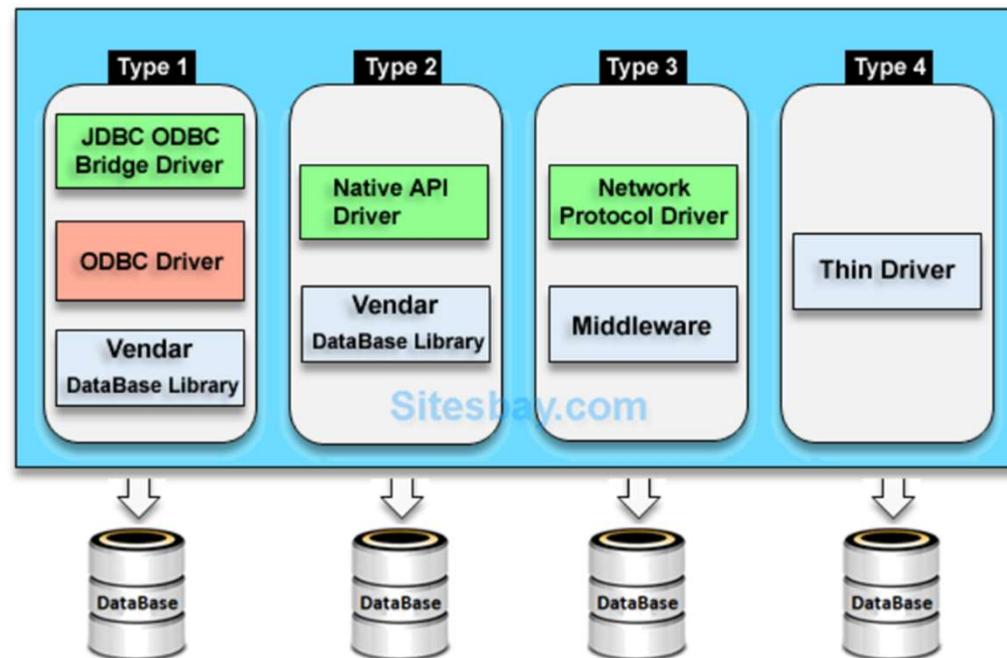
JDBC Driver is a software component that enables java application to communicate with the database.

There are 4 types of JDBC drivers, they are:

- Jdbc-Odbc Bridge Driver- **Type1**
- Native-API driver (partially java driver)-**Type 2**
- Network Protocol driver (fully java driver)-**Type 3**
- Thin driver (fully java driver)- **Type 4**

JAVA TECHNOLOGIES

JAVA Drivers



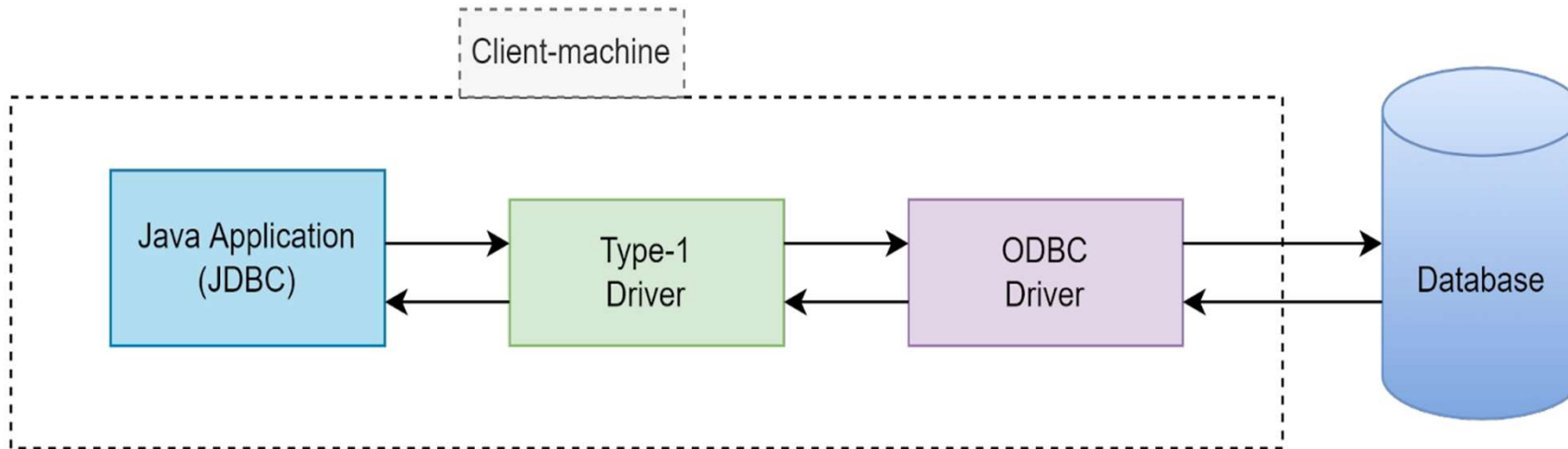
Type-1 Drivers

A type 1 driver, also known as a JDBC-ODBC bridge driver, acts as a middleman between Java applications and databases.

- Java application makes a JDBC call: Your Java program interacts with the database using JDBC methods like `getConnection` or `executeQuery`.
- Type 1 driver translates JDBC call to ODBC call: The driver converts these JDBC calls into ODBC function calls, the language understood by the ODBC driver.
- ODBC driver interacts with database: The ODBC driver, installed on the client machine, then communicates with the specific database using its native protocols.

JAVA TECHNOLOGIES

Type-1 Drivers



JAVA TECHNOLOGIES



Advantages

- It is easy to use and maintain. We do not have to separately install the Type-1 Driver because it is available as part of the JDK.
- The Type-1 Driver is database-independent. Type-1 Driver does not communicate with the database directly. Instead, the ODBC Driver does this work. Migrating from one database to another is easy as it is database-independent.

Limitations

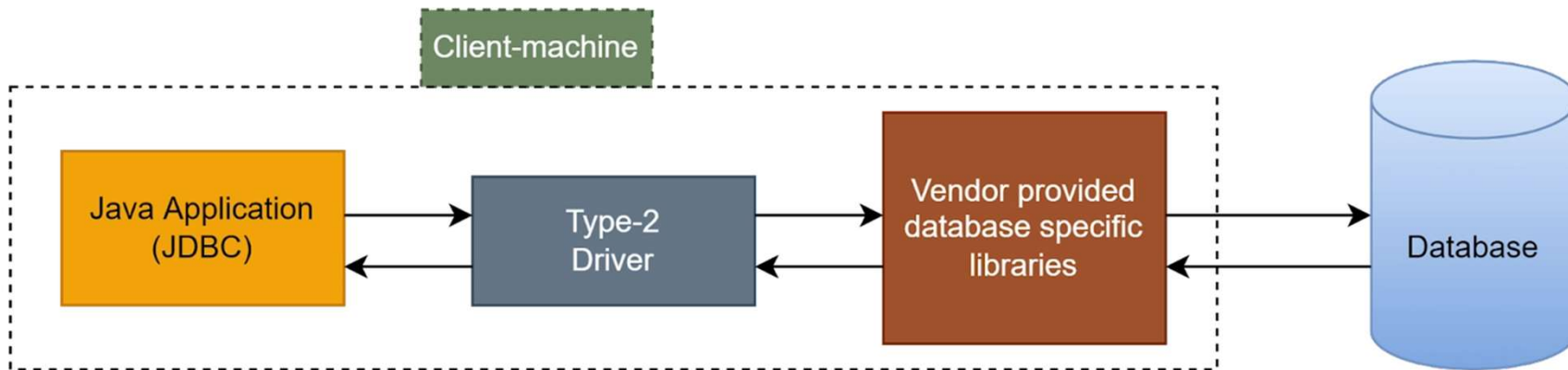
- It is the slowest Driver among all the Drivers because first, it converts the JDBC calls into the ODBC calls, and then the ODBC Driver converts the ODBC calls into the database-specific calls. This makes the overall process slow.
- The ODBC Driver is available for the Windows platform only. The Type-1 Driver depends internally on the ODBC Driver; hence it is platform dependent. No support from JDK 1.8 onwards.

Type-2 Drivers

- Type 2 drivers use a native API to communicate with a database system. Java native methods are used to invoke the API functions that perform database operations.
- Type 2 drivers are generally faster than Type 1 drivers.
- Type 2 drivers need native binary code installed and configured to work.
- You cannot use a Type 2 driver in an applet since applets cannot load native code.
- A Type 2 JDBC driver may require some Database Management System (DBMS) networking software to be installed.
- The Developer Kit for Java JDBC driver is a Type 2 JDBC driver.

JAVA TECHNOLOGIES

Type-2 Drivers- IBM DB2



Type-2 Drivers

Advantages

- It is faster as compared to the Type-1 Driver because it converts the function calls only once.
- ODBC Driver not required.
- It provides more portability.

Limitations

- It is a database-dependent Driver since it internally uses database-specific native libraries to interact with the database. Therefore migrating from one database to another is difficult.
- It is platform-dependent too.
- We have to mandatorily install the native libraries on the client machine or system.

JAVA TECHNOLOGIES

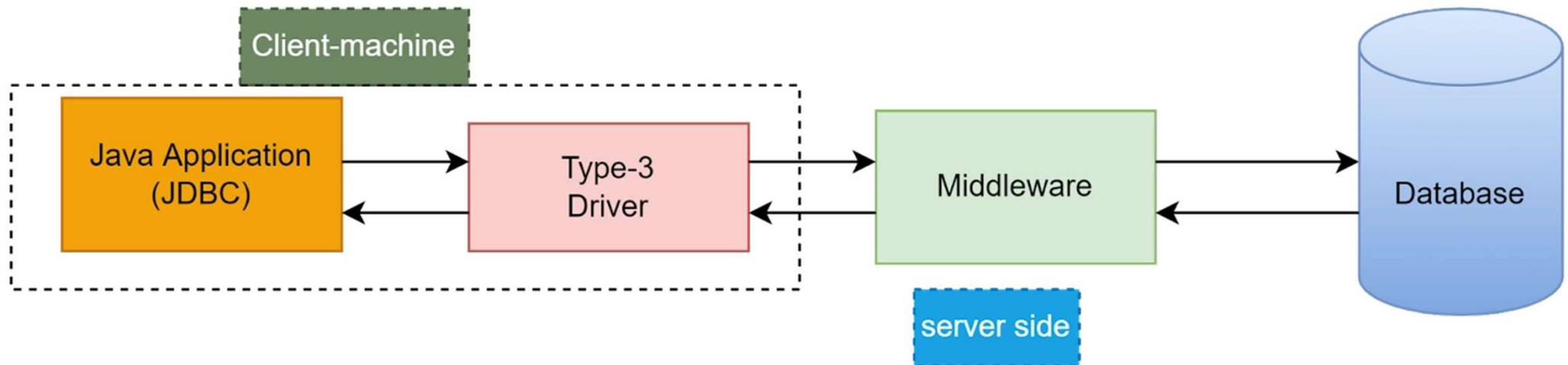


Type-3 Drivers

- These drivers use a networking protocol and middleware to communicate with a server.
- The server then translates the protocol to DBMS function calls specific to DBMS.
- Type 3 JDBC drivers are the most flexible JDBC solution because they do not require any native binary code on the client.
- A Type 3 driver does not need any client installation.

JAVA TECHNOLOGIES

Type-3 Drivers



Type-3 Drivers

Advantages

- It is database-independent.
- It is platform-independent as it is written in Java.
- The application server performs the necessary tasks; therefore, the client-side library is not required.

Limitations

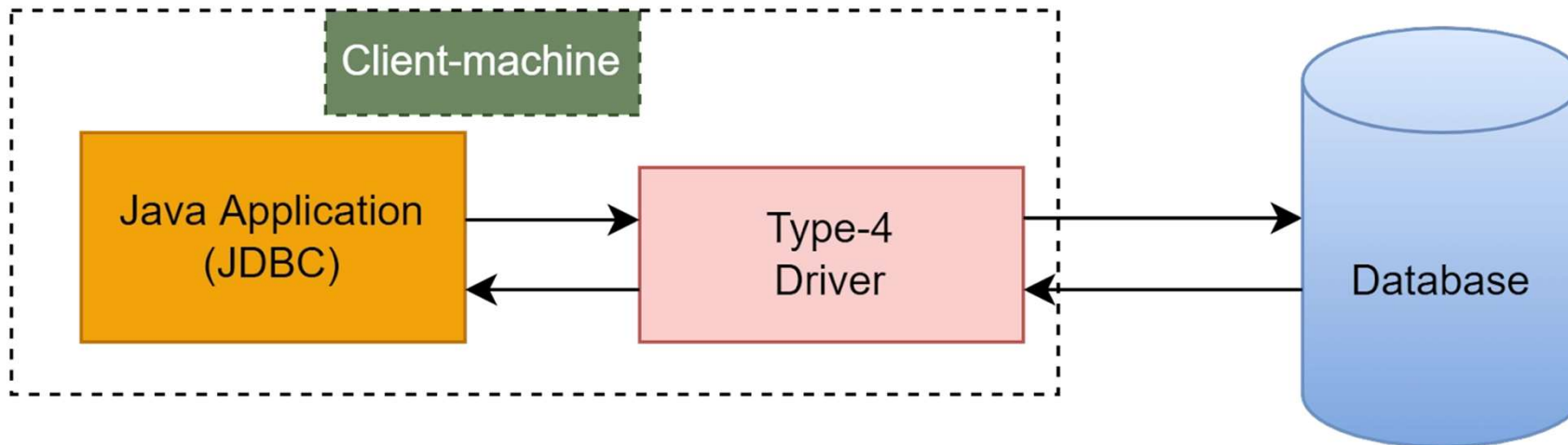
- It is expensive because of the maintenance of the network protocol Driver.
- We need network support on the client machine.

Type-4 Drivers

- A Type 4 driver uses Java to implement a DBMS vendor networking protocol.
- Since the protocols are usually proprietary, DBMS vendors are generally the only companies providing a Type 4 JDBC driver.
- Type 4 drivers are all Java drivers. This means that there is no client installation or configuration.
- However, a Type 4 driver may not be suitable for some applications if the underlying protocol does not handle issues such as security and network connectivity well.

JAVA TECHNOLOGIES

Type-4 Drivers



Type-4 Drivers

Advantages

- It is platform-independent.
- Since it communicates with the database directly, the overall performance is improved and faster.
- It does not require any native libraries or middleware servers.

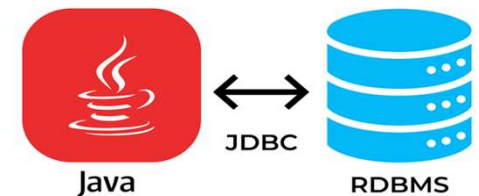
Limitation

- It is database-dependent.

JAVA TECHNOLOGIES

JDBC Connectivity:

- A real life application needs to manipulate data stored in a Database.
- A database is a collection of related data in the form of Tables. Most of the database uses SQL (Structured Query Language) to Insert, Delete, Update or retrieve stored data.
- In order to connect a Java application to a database designed in MySQL, Oracle, Sybase, MS SQL Server etc, you need a Bridge/Interface Driver Program.
- Java Provides JDBC (Java Database Connection)



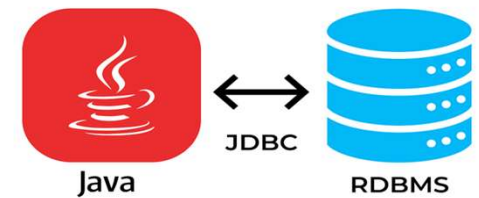
JAVA TECHNOLOGIES



SQL Databases

SQL (Structured Query Language) databases are relational databases that store data in tables with predefined schemas. SQL is used to manage and query data.

Common SQL databases include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.



JAVA TECHNOLOGIES

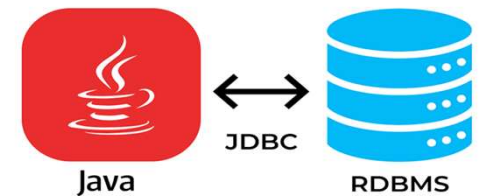


SQL Databases

SQL is the most widely used language for interacting with relational databases. It includes various sublanguages:

DDL (Data Definition Language): Used to define and manage database structure, including:

- **CREATE**: Create database objects (tables, indexes, views).
- **ALTER**: Modify existing database objects.
- **DROP**: Delete database objects.
- **TRUNCATE**: Remove all data from a table without removing the table itself.



JAVA TECHNOLOGIES

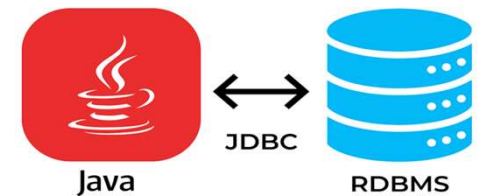


SQL Databases

DML (Data Manipulation Language):

Used for data manipulation, including:

- **SELECT**: Retrieve data from a database.
- **INSERT**: Add new data to a database.
- **UPDATE**: Modify existing data in a database.
- **DELETE**: Remove data from a database.



JAVA TECHNOLOGIES

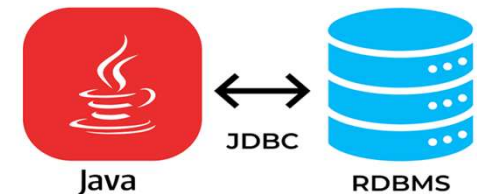
SQL Databases

DCL (Data Control Language): Used to control access to data, including:

- **GRANT:** Give users access privileges.
- **REVOKE:** Remove access privileges.

TCL (Transaction Control Language): Used to manage transactions, including:

- **COMMIT:** Save changes to the database.
- **ROLLBACK:** Undo changes to the database.
- **SAVEPOINT:** Create a point within a transaction to which you can roll back.



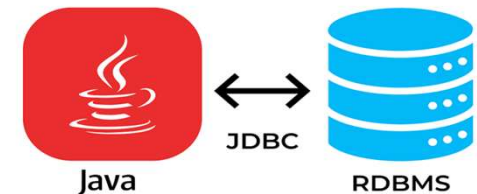
JAVA TECHNOLOGIES



JDBC Connectivity:

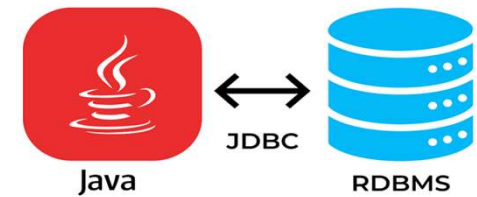
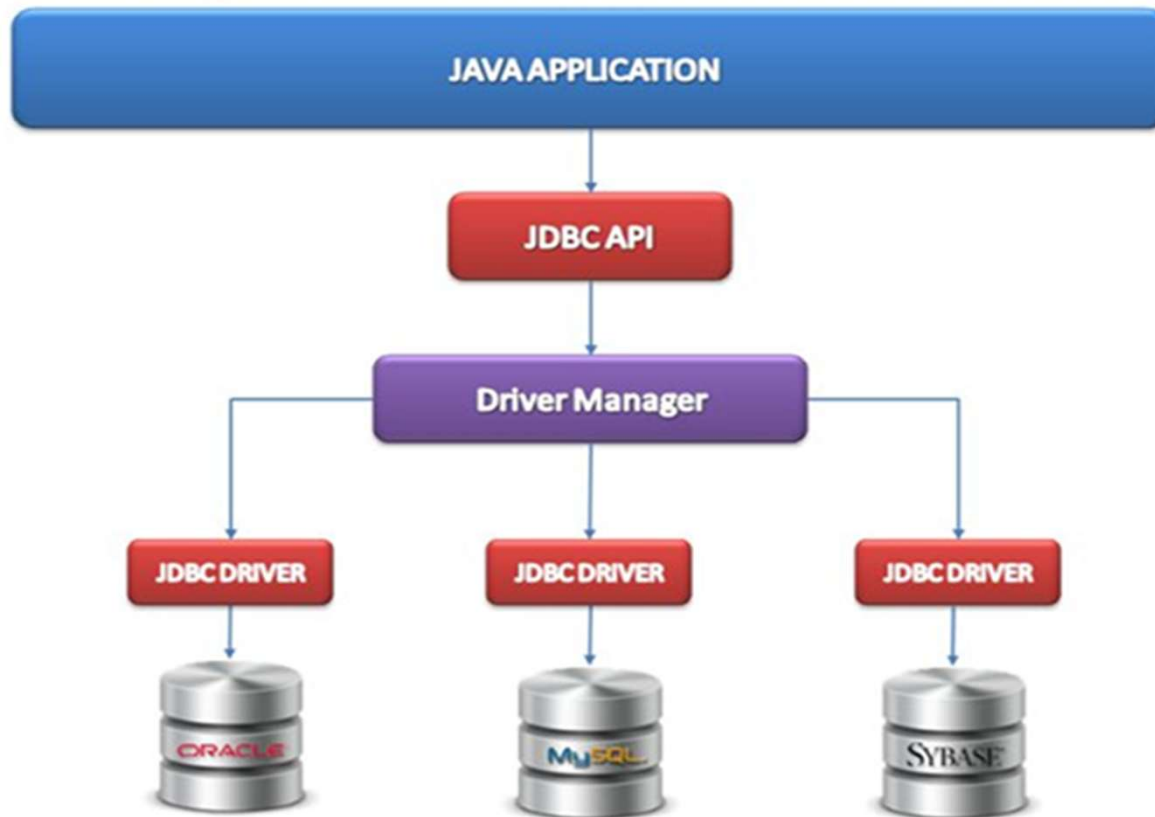
JDBC is JAVA's Database connection driver interface which performs the following task for the application.

- Establish a connection with a Database.
- Send SQL request (Query) to a Database Server.
- Returns Result obtained against Query.



JAVA TECHNOLOGIES

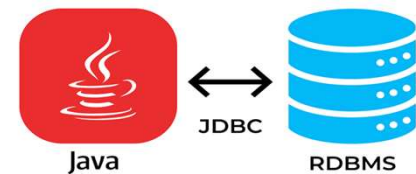
JDBC Architecture



JAVA TECHNOLOGIES

Main JDBC Classes and Interfaces:

1. **DriverManager (Class)**: Manages JDBC drivers and establishes connection with the database.
2. **SQLException (Class)**: Handles errors that occur during database operations.
3. **Driver (Interface)**: Provides the mechanism for connecting Java applications to a specific database.
4. **Connection (Interface)**: Represents an active connection between the Java program and the database.
5. **Statement (Interface)**: Used to execute static SQL queries.
6. **ResultSet (Interface)**: Stores and processes the results returned by a SQL query.



JAVA TECHNOLOGIES



1. Java Application

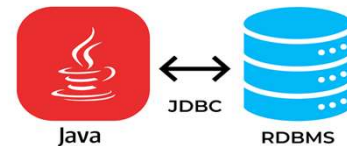
- This is the Java program written by the developer.
- It sends database requests such as INSERT, UPDATE, DELETE, and SELECT.
- The application uses JDBC API methods to communicate with the database.

2. JDBC API

- The JDBC API is a set of Java classes and interfaces used for database connectivity.
- It provides standard methods to interact with databases.

3. DriverManager

- DriverManager is a JDBC class responsible for managing database drivers.
- It selects the appropriate driver for the database.



JAVA TECHNOLOGIES

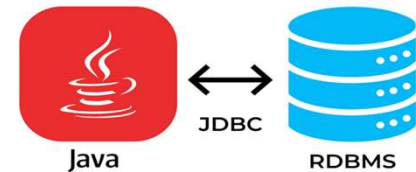
4. JDBC Driver

- A JDBC driver is a software component that converts JDBC calls into database-specific commands.
- Each database requires its own driver.

5. Database

The bottom layer represents actual databases such as:

- Oracle
- MySQL
- Sybase



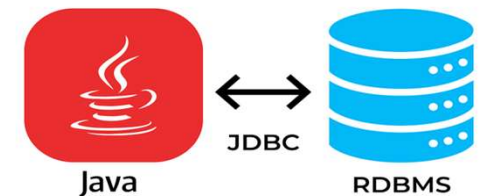
JAVA TECHNOLOGIES



SQL Databases

SQL (Structured Query Language) databases are relational databases that store data in tables with predefined schemas. SQL is used to manage and query data.

Common SQL databases include MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.



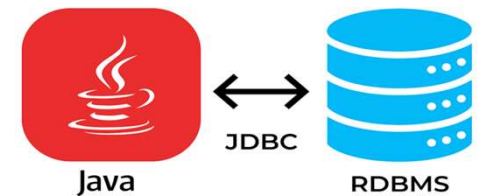
JAVA TECHNOLOGIES

SQL Databases

SQL is the most widely used language for interacting with relational databases. It includes various sublanguages:

DDL (Data Definition Language): Used to define and manage database structure, including:

- **CREATE**: Create database objects (tables, indexes, views).
- **ALTER**: Modify existing database objects.
- **DROP**: Delete database objects.
- **TRUNCATE**: Remove all data from a table without removing the table itself.



JAVA TECHNOLOGIES

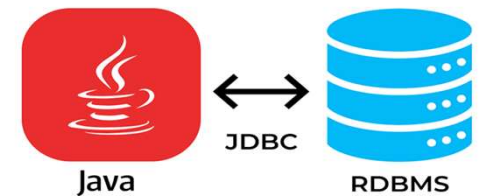


SQL Databases

DML (Data Manipulation Language):

Used for data manipulation, including:

- **SELECT**: Retrieve data from a database.
- **INSERT**: Add new data to a database.
- **UPDATE**: Modify existing data in a database.
- **DELETE**: Remove data from a database.



JAVA TECHNOLOGIES

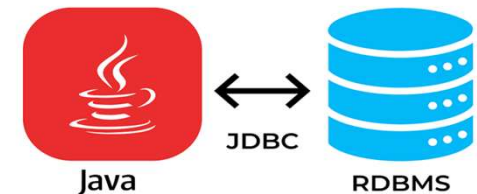
SQL Databases

DCL (Data Control Language): Used to control access to data, including:

- **GRANT:** Give users access privileges.
- **REVOKE:** Remove access privileges.

TCL (Transaction Control Language): Used to manage transactions, including:

- **COMMIT:** Save changes to the database.
- **ROLLBACK:** Undo changes to the database.
- **SAVEPOINT:** Create a point within a transaction to which you can roll back.



JAVA TECHNOLOGIES



Understanding CRUD Operation

CRUD operations in a database refer to the four basic operations performed on data in a table.

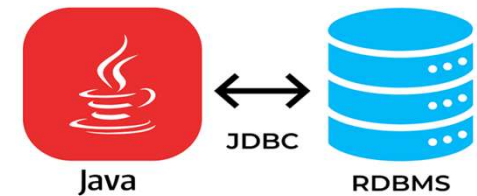
C – Create

R – Read

U – Update

D – Delete

These operations are performed using **SQL queries**.



JAVA TECHNOLOGIES



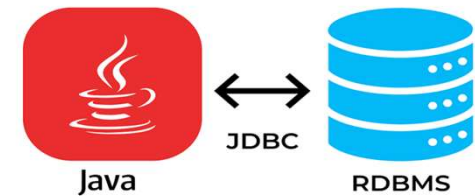
Understanding CRUD Operation

1. Create Operation (INSERT)

The INSERT query is used to add new records to a table.

Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```



JAVA TECHNOLOGIES



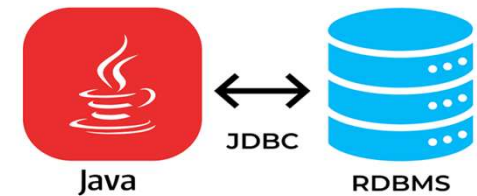
Understanding CRUD Operation

2. Read Operation (SELECT)

The **SELECT** query is used to **retrieve data from a table**.

Syntax

```
SELECT column_name FROM table_name;
```



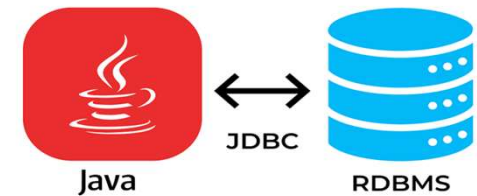
Understanding CRUD Operation

3. Update Operation (UPDATE)

The **UPDATE** query is used to **modify existing records** in a table.

Syntax

```
UPDATE table_name  
SET column1 = value1  
WHERE condition;
```



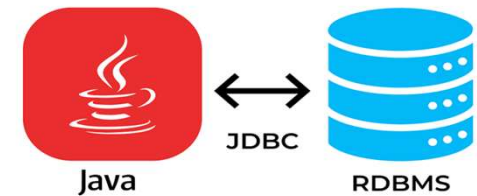
Understanding CRUD Operation

4. Delete Operation (DELETE)

The **DELETE** query is used to **remove records from a table**.

Syntax

```
DELETE FROM table_name  
WHERE condition;
```



CRUD Example

Design a database table named Student to store student details such as ID, Name, and Marks.

1. Write SQL queries to perform the following CRUD operations:
2. Create the Student table with the attributes:
 - id (Primary Key)
 - name
 - marks
3. Insert at least two student records into the table.
4. Display all the records from the Student table.
5. Update the marks of a particular student using their ID.
6. Delete a student record from the table using their ID.
7. The program should demonstrate all four database operations: Create, Read, Update, and Delete (CRUD) using SQL queries.

CRUD Example

Create Table (C – Create Table)

```
CREATE TABLE Student (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    marks INT  
);
```

JAVA TECHNOLOGIES



CRUD Example

Insert Records (Create Operation)

```
INSERT INTO Student (id, name, marks)  
VALUES (101, 'Ravi', 85);
```

```
INSERT INTO Student (id, name, marks)  
VALUES (102, 'Anita', 90);
```

JAVA TECHNOLOGIES



CRUD Example

Read Records (R – Read Operation)

Display all records from the table.

```
SELECT * FROM Student;
```

CRUD Example

Read Records (R – Read Operation)

Display all records from the table.

```
SELECT * FROM Student;
```

id	name	marks
101	Ravi	85
102	Anita	90

CRUD Example

Update Record (U – Update Operation)

Update marks of student with ID **101**.

```
UPDATE Student  
SET marks = 95  
WHERE id = 101;
```

id	name	marks
101	Ravi	95
102	Anita	90

CRUD Example

Delete Record (D – Delete Operation)

Delete student with ID **102**.

```
DELETE FROM Student  
WHERE id = 102;
```

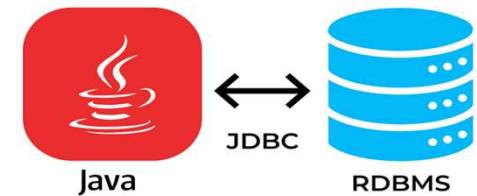
id	name	marks
101	Ravi	95

JAVA TECHNOLOGIES



Importance of JDBC API

JDBC (Java Database Connectivity) API is used to connect Java applications with databases like MySQL, Oracle, SQL Server, etc. It allows Java programs to execute SQL queries and interact with the database.



JAVA TECHNOLOGIES

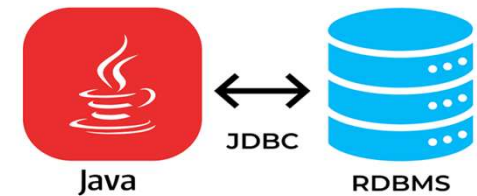


Importance of JDBC API

1. Database Connectivity

JDBC API provides a standard way to connect Java programs to databases.

It establishes a connection between the Java application and the database server.



JAVA TECHNOLOGIES

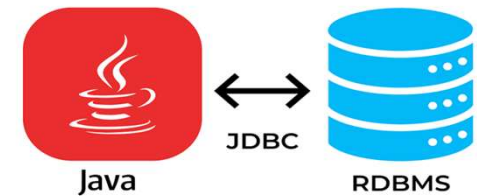


Importance of JDBC API

1. Database Connectivity

JDBC API provides a standard way to connect Java programs to databases.

It establishes a connection between the Java application and the database server.



JAVA TECHNOLOGIES

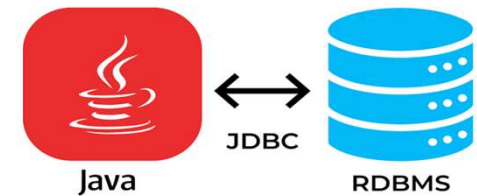


Importance of JDBC API

2. It allows execution of SQL statements such as:

- SELECT
- INSERT
- UPDATE
- DELETE

This helps perform CRUD operations on the database.



JAVA TECHNOLOGIES

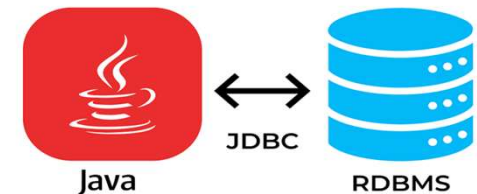


Importance of JDBC API

3. Database Independence

JDBC provides a common interface for different databases.

A Java program written using JDBC can work with MySQL, Oracle, PostgreSQL, etc. with minimal changes.

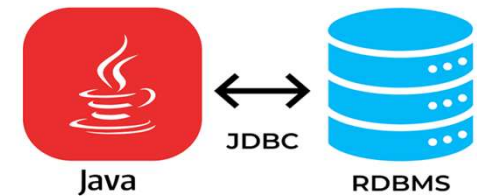


Importance of JDBC API

4. Data Retrieval and Manipulation

JDBC allows programs to **retrieve data from the database and modify it** using classes like:

- Connection
- Statement
- PreparedStatement
- ResultSet



JAVA TECHNOLOGIES



Importance of JDBC API

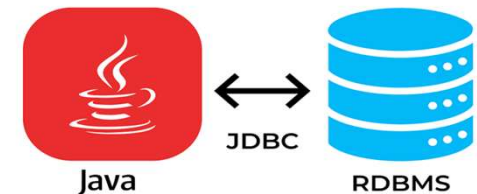
5. Secure Database Access

Using PreparedStatement, JDBC helps prevent SQL injection and improves security.

6. Used in Enterprise Applications

JDBC is widely used in:

- Web applications
- Banking systems
- E-commerce systems
- Student management systems



JDBC Connectivity:

Installation:

Install MySQL Server

Download MySQL from: <https://dev.mysql.com/downloads/mysql/>

Install it and remember:

Username: root

Password: (set during installation)

JDBC Connectivity:

Login to MySQL and create a sample database.

- `CREATE DATABASE college;`
- `USE college`
- `CREATE TABLE student(
id INT PRIMARY KEY,
name VARCHAR(50),
marks INT
);`

JDBC Connectivity:

Download MySQL JDBC Driver

Download **MySQL Connector/J**

<https://dev.mysql.com/downloads/connector/j/>

After downloading:

- Extract the zip file
- You will see file like

mysql-connector-j-8.3.0.jar

JDBC Connectivity:

Download MySQL JDBC Driver

Download **MySQL Connector/J**

<https://dev.mysql.com/downloads/connector/j/>

After downloading:

- Extract the zip file
- You will see file like

mysql-connector-j-8.3.0.jar

JDBC Connectivity:

Add JAR to Java Project (Important)

- Open **Explorer** panel
- Expand **JAVA PROJECTS** section
- Under **Referenced Libraries**

Click +

Then select:

mysql-connector-j-8.3.0.jar from the extracted folder

Now the library is added.

JAVA TECHNOLOGIES

JDBC Connectivity:

Write the Java Code for JDBC connectivity and run the program.

JAVA TECHNOLOGIES

Program : Demonstration on JDBC Connection

```
package UNIT_3;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class JDBC_prg1 {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/college";
        String username = "root";
        String password = "root"; // change to your MySQL password
```

Program

```
try {
```

```
// Load Driver (optional in new versions, but good for exams)
```

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

```
// Establish Connection
```

```
Connection con = DriverManager.getConnection(url, username, password);
```

```
System.out.println("Connected Successfully!");
```

```
// Create Statement
```

```
Statement stmt = con.createStatement();
```


Program

// Insert Data

```
String sql = "INSERT INTO student VALUES (1, 'Sam', 90)";  
stmt.executeUpdate(sql);  
System.out.println("Data Inserted Successfully!");
```

// Close Connection

```
con.close();
```

```
} catch (Exception e) {  
    System.out.println("Error: " + e);  
}  
} }
```

Program

Output:

Connected Successfully!
Data Inserted Successfully!

Open MySQL: Check table student for insert operation.

```
mysql> select * from student;
+----+-----+-----+
| id | name | marks |
+----+-----+-----+
|  1 | Sam  |    90 |
+----+-----+-----+
1 row in set (0.00 sec)
```



PES
UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Samyukta D Kumta

Department of Computer Applications

samyuktad@pes.edu